

Shield Gates

A Multi-Layered Secure Email Gateway with Integrated ML, NLP, and Sandbox Analysis

Rama Abu-Haweleha Rana Shiqer

Department of Computer Science — Networks and Information Security
An-Najah National University, Nablus, Palestine

Supervised by: Dr. Othman M. Othman & A. Dr. Prof. Jehad M. Hamamreh

Abstract

Email remains one of the primary vectors for phishing, malware delivery, and social engineering attacks in modern organisations. This paper presents Shield Gates, a production-grade Secure Email Gateway (SEG) that positions a dedicated filtering layer between external SMTP traffic and the internal mail infrastructure. The system implements a three-track detection pipeline: (Track A) header authentication analysis using XGBoost with SPF, DKIM, and DMARC signals; (Track B) body content analysis combining a fine-tuned RoBERTa transformer and a TF-IDF fallback classifier; and (Track C) dynamic attachment analysis via CAPEv2 sandboxing across isolated Windows and Linux virtual machines. A central Decision Engine aggregates risk scores from all three tracks using adaptive signal weighting across 50+ features and 28 deterministic hard rules. Evaluated on a balanced dataset of 14,512 emails, the fused model achieved a validation ROC-AUC of 0.9984, a phishing Recall of 96.42%, and a False Positive Rate of only 1.64%, confirming its readiness for institutional deployment.

Keywords:

Email Security, Phishing Detection, Mail Gateway, XGBoost, RoBERTa, TF-IDF, CAPEv2, Sandboxing, SPF/DKIM/DMARC, NLP, Decision Engine

1. Introduction

Email is the dominant communication channel in both academic and enterprise environments, making it the most frequently exploited attack surface for credential theft, malware distribution, and social engineering. The Anti-Phishing Working Group (APWG) reported over 1.35 million unique phishing sites observed in 2023, with email remaining the primary delivery mechanism. Traditional perimeter defences — firewalls, rule-based spam filters, and signature-based antivirus — are increasingly insufficient against modern evasion techniques such as homograph domain attacks, obfuscated JavaScript payloads, and phishing content hosted on high-reputation cloud platforms.

This paper presents **Shield Gates**, a secure email gateway designed to close this detection gap through a multi-layered, AI-driven approach. Rather than relying on a single detection signal, the system fuses structural authentication signals, deep natural language understanding of message bodies, and behavioural analysis of executed attachments into a single unified risk score. The gateway intercepts all inbound SMTP traffic through a Postfix content-filter pipeline, routes each message through three specialised analysis tracks, and applies a deterministic Decision Engine to produce a binary DROP / DELIVER verdict before any email reaches the internal mail server.

1.1 Problem Statement

- **Phishing and Credential Harvesting:** Attackers craft deceptive messages that mimic legitimate institutions to extract sensitive credentials.
- **Malware-Laden Attachments:** Executables, office macros, and archive files are used to deliver ransomware and RATs.
- **Authentication Spoofing:** Sender domains are forged or typosquatted, bypassing naive domain-name checks.
- **Detection Evasion:** Phishing content increasingly hosted on trusted CDNs and cloud storage eliminates URL-reputation-based blocking.
- **Server Availability:** Without a spooling layer, mail server downtime causes permanent message loss.

1.2 Objectives

- Deploy an inline SMTP proxy that intercepts mail before internal delivery.
- Implement a three-track detection pipeline covering header metadata, body content, and attachments.
- Achieve a phishing recall $\geq 95\%$ with a false positive rate $\leq 3\%$ on a balanced, production-representative dataset.
- Provide dynamic malware detonation through isolated CAPEv2 sandboxes with automatic snapshot restoration.
- Operate as a fail-closed system: default to DROP on any processing error.

2. Related Work

2.1 Commercial Email Security Gateways

Proofpoint Email Protection is one of the leading commercial SEG platforms, employing classification-based mechanisms to filter spam, phishing, and malware with confusion-matrix metrics used for quality assurance across threat categories including bulk mail, impostor messages, and malware-laden attachments [1]. Its modular pipeline separates sender reputation checks from content inspection, an architectural principle that Shield Gates adopts and extends with deep learning models.

Cisco Secure Email Gateway (formerly IronPort) combines reputation filtering, anti-spam engines, and sandboxing in a hardware-software hybrid appliance. While highly effective in enterprise deployments, its closed-source nature and licensing costs make it inaccessible for academic and SME environments — the primary deployment target of Shield Gates.

2.2 Academic Research Landscape

Research in ML-based email filtering has progressed from bag-of-words Naive Bayes classifiers toward transformer-based models. Studies have demonstrated that BERT-family models significantly outperform TF-IDF approaches on phishing body detection, particularly for zero-day campaigns that lack keyword overlap with historical training data. However, most published systems address only one detection layer — either textual or structural — and lack integration with dynamic sandboxing for attachment analysis.

Dynamic malware analysis through sandbox environments (Cuckoo Sandbox, Any.Run, Joe Sandbox) has established the gold standard for attachment-level threat intelligence. CAPEv2, the platform chosen for Shield Gates, extends Cuckoo with advanced memory forensics and automated configuration/payload extraction, making it particularly effective against packed and polymorphic malware.

2.3 Positioning of Shield Gates

Shield Gates differentiates itself by being the first fully open-source, academically documented system to integrate all three detection layers — header ML classification, transformer-based body NLP, and live CAPEv2 sandboxing — into a single Postfix-anchored gateway with a unified decision engine. This end-to-end integration on commodity hardware makes it reproducible and deployable in institutional network environments.

3. System Architecture

3.1 High-Level Design

Shield Gates is deployed as an inline SMTP proxy sitting between the external internet (192.168.200.0/24 NATNetwork) and the internal mail server (10.10.0.20). All inbound TCP connections on port 25 are accepted by Postfix on the Mail Gateway, which immediately forwards each message to **mail_filter.py** on port 10025 via a content-filter directive. After analysis, clean messages are re-injected through port 10026, which bypasses further content filtering to prevent processing loops. A sandbox network (10.0.0.0/24) provides a fully isolated environment for attachment detonation.

Component	Role	Address / Port
Mail Gateway (MGW)	SMTP interception, filtering pipeline orchestration	192.168.200.254 / 10.10.0.10
Internal Mail Server	Final delivery — Postfix + Dovecot (IMAP/POP3)	10.10.0.20
Windows Sandbox (CAPE_WIN)	Execution of Windows executables/macros	10.0.0.30
Linux Sandbox (CAPE_Linux)	Execution of ELF, shell scripts, .deb/.rpm	10.0.0.20
CAPEv2 Master (on MGW)	Sandbox orchestration, report aggregation	127.0.0.1:8000
mail_filter.py	Central SMTP proxy and pipeline controller	127.0.0.1:10025
Decision Engine	Risk aggregation and DROP/DELIVER verdict	Internal Python module

3.2 Network Topology

Three isolated network segments are used. The **NAT network** (192.168.200.0/24) connects external clients to the MGW. The **internal network** (10.10.0.0/24) connects the MGW to the mail server. The **sandbox network** (10.0.0.0/24) connects the MGW's CAPEv2 master to the Windows and Linux sandbox VMs. IP forwarding and NAT masquerading are configured on the MGW using iptables, and the sandbox network has no route to the production network, enforcing complete isolation of detonated malware.

3.3 Mail Transport Layer

Postfix is configured with two distinct listener contexts. Port 587 (submission) serves Thunderbird mail clients with SASL authentication delegated to Dovecot on the internal mail server via TCP to port 12345. Port 25 (smtp) accepts inbound relay mail. Both contexts apply the content-filter directive routing to **mail_filter.py**. The re-injection listener on port 10026 accepts only trusted mynetworks ranges and has content_filter= (empty) to prevent loops. Postfix on the internal mail server handles final delivery to Maildir-format mailboxes, which are accessible to Thunderbird clients via IMAP on port 143.

3.4 Service Management

Both mail_filter.py and the CAPEv2 scheduler/web services are managed as systemd units with Restart=always, ensuring automatic recovery from crashes. The mail-filter.service depends on postfix.service, maintaining correct startup ordering. The CAPEv2 cape-scheduler.service depends on mongod.service, as the MongoDB database is required for task queuing and result storage.

4. Methodology and Detection Pipeline

The Shield Gates detection pipeline is implemented in five Python modules: **extract_phishing_features.py** (feature extraction), **header.py** (Track A), **body.py** (Track B), **sandbox_client.py** (Track C), and **decision_engine.py** (risk aggregation). Each module is independently loadable and testable. The main **mail_filter.py** orchestrates these modules within an asynchronous SMTP handler built on Python's asyncio framework.

4.1 Email Parsing and Feature Extraction

Upon receiving an SMTP DATA segment, **mail_filter.py** parses the raw message using Python's email library and passes the structured object to **extract_phishing_features.py**, which operates two parallel processing tracks:

Track A — Semantic Feature Extraction (for ML models)

This track produces 123 structured features across seven categories without modifying the original email content. Feature categories include: header authentication signals (SPF, DKIM, DMARC pass/fail flags, domain mismatch, display name spoofing), URL characteristics (IP-based URLs, mismatch between anchor text and href domains, suspicious TLDs, redirect chains, encoding evasion), body statistical features (uppercase ratio, word count, punctuation density), phishing keyword scores (12 high-signal keywords), psychological manipulation scores (urgency, fear, curiosity), HTML structure indicators (`eval()`, `iframe`, `form`, data URIs, JavaScript), and attachment metadata (file type, double extension detection).

Track B — Structural Preprocessing (for NLP models)

This track produces a clean, normalised version of the email body suitable for transformer input by removing HTML tags via BeautifulSoup, decoding HTML entities, and replacing variable content (URLs, email addresses, monetary values, IP addresses) with standardised tokens (`URL_TOKEN`, `EMAIL_TOKEN`, etc.). This token-substitution approach allows the NLP model to learn structural phishing patterns rather than memorise specific domain names or dollar amounts.

4.2 Track A — Header Analysis (XGBoost Classifier)

header.py receives the 123-feature vector extracted in Track A and applies an XGBoost gradient boosting classifier trained on 14,512 labelled emails. The model is loaded from disk on first invocation; if absent, it trains from the available dataset using a 60/20/20 split. The XGBoost ensemble aggregates predictions across 300 decision trees, and the final **header_model_risk** score is produced via `predict_proba()`, yielding a continuous value in [0, 1]. If no trained model and no dataset are available, the module falls back to a weighted heuristic that sums feature contributions normalised through a tanh function, ensuring the pipeline never halts. The header risk score carries a signal weight of 1.50 in the composite scoring formula.

4.3 Track B — Body NLP Analysis (RoBERTa + TF-IDF)

body.py implements a cascading model hierarchy. The primary model is a fine-tuned **RoBERTa** transformer loaded from the `roberta_finetuned/` directory. If the fine-tuned model is unavailable, a base roberta-base model is loaded and on-the-fly fine-tuning is attempted using the available labelled

dataset. If the RoBERTa tokenizer cannot be instantiated (e.g., due to missing sentencepiece), the system falls back to a **TF-IDF + Logistic Regression** model stored as `tfidf_body_model.pkl`. As a last resort, a semantic-only score is computed from the metadata feature weights.

The final **body_model_risk** score is a fusion: if an NLP probability is available, 55% weight is assigned to the NLP model output and 45% to the semantic metadata score. This ensures that both linguistic patterns and technical phishing signals are captured. The body risk score carries the highest individual model weight of 1.80 in the composite formula.

4.4 Track C — Dynamic Attachment Analysis (CAPEv2)

When an email attachment is detected, it is written to an isolated directory keyed by the email's Message-ID and submitted to the CAPEv2 Web API (<http://127.0.0.1:8000>) via `sandbox_client.py`. The submission logic implements extension-based routing:

- **Windows executables** (.exe, .msi, .bat, .cmd, .lnk, .dll, .docm, .xls, .ps1): routed to CAPE_WIN (10.0.0.30).
- **Linux executables** (.elf, .sh, .bin, .deb, .rpm): routed to CAPE_Linux (10.0.0.20).
- **Cross-platform** (.pdf, .zip, .7z, .iso): submitted to both sandboxes simultaneously.

CAPEv2 executes the sample in the guest VM, monitors API calls, network traffic (PCAP), file system modifications, and registry changes for a configurable timeout (120 s default). After execution, the guest VM is automatically restored to a clean snapshot state (CAPE-Clean), ensuring each subsequent sample is analysed in an untainted environment. The sandbox client implements exponential-backoff polling and returns a **risk_score** in [0, 1] derived from the CAPEv2 `malscore`. Failed or timed-out tasks return 0.5 to avoid false-clean verdicts (fail-closed principle). An attachment sandbox `risk_score` ≥ 0.70 triggers an immediate **R-VETO** DROP before composite scoring.

4.5 Decision Engine

`decision_engine.py` is the final arbitration layer. It implements a three-stage process:

Stage 1 — Sandbox Veto

If any attachment's `sandbox risk_score` ≥ 0.70 , the email is immediately dropped regardless of composite score or any other signal. This hard veto ensures that confirmed malware attachments are never delivered.

Stage 2 — Composite Scoring

A weighted sum is computed across all active signals: $\text{Composite} = \frac{\sum(\text{signal_value} \times \text{weight})}{\sum(\text{weights})}$. Signal weights range from 0.05 (`total_phishing_keywords`) to 2.50 (`cape_malscore_norm`, `cape_process_injection`). If `composite` ≥ 0.55 (equivalent to 5.5/10), the verdict is DROP.

Stage 3 — Deterministic Hard Rules

28 rule conditions (R-D01 through R-D28, plus R-C01 for model contradiction) are evaluated after scoring. Each rule targets a specific attack pattern — e.g., R-D05 fires when a URL contains an IP address AND a password-input form is detected; R-D21 fires when all three of `urgency_score` > 0.60 , `fear_score` > 0.60 , and ≥ 5 unique phishing keywords are present simultaneously. Any fired rule forces

DROP independently of the composite score. Contradiction rule R-C01 fires when the header model and body model diverge severely (HIGH contradiction severity), indicating possible targeted evasion.

Key Decision Thresholds

Threshold	Value	Purpose
T_DROP / T_DELIVER	0.55 (5.5/10)	Composite score at or above → DROP
T_CAPE_HARD (R-D01)	0.70 (7.0/10)	Confirmed malware malscore → instant DROP
T_CAPE_SOFT (R-D13)	0.40–0.70	Suspicious sandbox range → DROP
SANDBOX_VETO	0.70 (7.0/10)	Per-attachment risk_score → pre-scoring DROP
Body model high	> 0.70	Divergence rules R-D14 / R-D15
Urgency + Fear + Keywords	> 0.60 / > 0.60 / ≥ 5	Social engineering rule R-D21

5. Dataset Engineering and Experimental Setup

5.1 Dataset Construction

The initial dataset compiled via `build_dataset.py` yielded 2,909 samples with a severe class imbalance: 408 phishing emails (14.0%) vs. 2,501 legitimate (Ham) emails (86.0%). Training on such a skewed distribution would have produced the accuracy paradox — high overall accuracy driven by majority-class bias with a dangerously high false negative rate.

A systematic augmentation pipeline (`append_dataset.py`) was implemented. Phishing samples were expanded using the **Nazario Phishing Corpus** (URL-based attacks and obfuscated text), raising the phishing count to 8,109. Legitimate samples were augmented with the **Apache SpamAssassin MailingList** corpus and the **Enron Corporate** email dataset, expanding the Ham pool to 6,403. The final dataset contains **14,512 rows** with a near-balanced distribution of 55.9% phishing / 44.1% Ham.

A source stratification column was added to track dataset provenance across four streams: `Nazario_Url_Attack` (7,559), `SpamAssassin_MailingList` (6,113), `Nazario_Text_Obfuscated` (550), and `Enron_Corporate` (290). This enables per-source performance monitoring and prevents any single noisy corpus from distorting the model's overall precision.

An additional algorithmic safety net was applied via XGBoost's `scale_pos_weight` hyperparameter, computed as `Ham_count / Phishing_count`. This dynamically adjusts the gradient penalty for the minority class, preventing the model from over-fitting to the majority class even if class ratios shift in future updates.

5.2 Data Partitioning

Split	Proportion	Samples	Purpose
Training	60%	8,706	XGBoost and RoBERTa parameter optimisation
Validation	20%	2,902	Threshold tuning, error analysis, hyperparameter selection
Testing	20%	2,902	Final held-out benchmark (unseen during training)

5.3 Evaluation Metrics

Given the security context — where an undetected phishing email (False Negative) is more harmful than a falsely flagged legitimate email (False Positive) — the primary metric is **Recall (Sensitivity)**. Secondary metrics are **Precision**, **F1-Score**, **ROC-AUC**, and **Average Precision (PR-AUC)**. The optimal decision threshold was determined programmatically using the **Youden-J index** ($J = \text{Sensitivity} + \text{Specificity} - 1$) applied to the validation ROC curve, yielding a final production threshold of **0.3660**.

6. Results and Performance Analysis

6.1 Cross-Stage Model Performance

The fused XGBoost header model was evaluated across all three dataset splits. Results are summarised in Table 2.

Stage	Split %	Samples	ROC-AUC	F1-Score	Precision	Recall
Training	60%	8,706	0.9994	0.9851	0.9935	0.9768
Testing	20%	2,902	0.9986	0.9735	0.9836	0.9636
Validation	20%	2,902	0.9984	0.9754	0.9868	0.9642

Table 2: Cross-Stage Model Performance

6.2 Generalisation and Overfitting Analysis

The generalisation gap — the difference in ROC-AUC between training and validation — is 0.0010 (0.9994 → 0.9984). This near-zero variance confirms that the combination of physical dataset balancing and algorithmic scale_pos_weight completely immunised the classifiers against overfitting despite a 5× increase in dataset size and a 41.9-point shift in the phishing class ratio.

6.3 Operational Metrics on Validation Set

The validation set contained 1,280 legitimate emails and 1,622 phishing emails. The gateway produced:

- **True Positives (phishing correctly blocked):** 1,564
- **False Negatives (phishing missed):** 58 → Miss Rate = 3.58%
- **True Negatives (legitimate correctly delivered):** 1,259
- **False Positives (legitimate incorrectly blocked):** 21 → FPR = 1.64%
- **Phishing Recall:** 96.42%
- **Precision:** 98.68%

6.4 Discriminative Power — ROC and PR Curves

The Receiver Operating Characteristic (ROC) curve achieves a consistent AUC of 0.9984 on the validation split, confirming near-perfect class separation across all threshold boundaries. The Precision-Recall curve yields an Average Precision (AP) of 0.9988, maintaining high precision even at very high recall levels. This is especially significant for email security deployments where high-velocity phishing campaigns must be intercepted without excessive false alarm rates.

6.5 Feature Importance Analysis

XGBoost feature importance (information gain) identified the following top predictors, ordered by their contribution to the model's decision boundaries:

Rank	Feature	Gain	Interpretation
1	url_mismatch_count	0.1870	Anchor text / href domain mismatch — strongest phishing signal
2	url_has_ip	0.1278	URL uses raw IP instead of domain name
3	domain_mismatch	0.1210	From display domain differs from SMTP envelope domain
4	received_hops	0.0838	Excessive routing hops — obfuscated relay chains
5	has_cc	0.0682	CC field presence — spam broadcast pattern

Rank	Feature	Gain	Interpretation
6	subject_exclamation	0.0561	Exclamation marks in subject — urgency signalling
7	has_reply_to	0.0553	Reply-to mismatch — credential harvesting pattern
8	subject_all_caps	0.0538	Full caps subject — psychological pressure
9	url_count	0.0485	Total URL volume
10	has_dkim	0.0315	DKIM header presence (absence correlates with phishing)

Table 3: Top 10 XGBoost Header Model Features by Information Gain

The dominance of URL-based signals (ranks 1, 2, 9) alongside authentication failures (ranks 3, 10) and routing anomalies (rank 4) validates the multi-feature design. No single feature is individually sufficient; the ensemble's strength derives from combining these complementary signals.

6.6 End-to-End System Test

A live phishing simulation was conducted by injecting a raw SMTP session via telnet from an attacker VM (192.168.200.100) targeting the MGW on port 25. The test email contained an urgency-laden subject line, a malicious <http://fake-winner.xyz/claim> URL, and classic social engineering language. The pipeline returned:

- Header Analyzer risk: **0.58** (moderate — suspicious sender, no authentication)
- Body Analyzer risk: **0.83** (high — via TF-IDF + semantic fusion after RoBERTa tokenizer fallback)
- Combined score: **0.8317** $\geq$ 0.55 threshold → **DROPPED**
- Postfix log: message ID 2F942160D4B rejected and removed from queue.

7. Error Analysis and Optimisation

7.1 False Positive Profile Analysis

A post-training error analysis script (`error_analysis.py`) isolated 643 False Positive boundary cases. Examination of the top FP samples (rows 425, 444, 467) revealed a consistent structural pattern: each message recorded exactly 6 Received headers (`received_hops = 6`) and a `subject_caps_ratio` between 0.13 and 0.18. In practical infrastructures, 6-hop routing is characteristic of legitimate corporate distribution-list or mailing-list relay chains. The XGBoost classifier incorrectly associated this routing depth with the suspicious relay obfuscation pattern used by bulk phishers. Mitigation involves adding a domain-reputation contextual signal to differentiate mailing-list MTAs from anonymising relay chains.

7.2 False Negative Profile Analysis

3,187 False Negative boundary cases were isolated. The most significant pattern — exemplified by row 2 — involved a `url_count` of 131 while maintaining `url_suspicious_tlds = 0`. This indicates a sophisticated evasion technique: the attacker hosted all malicious content on high-reputation domains (mainstream cloud storage or compromised business sites), eliminating any TLD-based signal. None of the 131 URLs triggered the `url_has_ip` or `url_suspicious_tlds` features, causing the composite score to fall below the DROP threshold.

Proposed mitigations include: (1) integrating real-time URL reputation lookup against threat intelligence feeds (VirusTotal, OpenPhish); (2) applying the CAPEv2 sandbox to URL-embedded redirects in addition to file attachments; and (3) training a dedicated URL-sequence model to detect anomalous link density patterns independent of TLD reputation.

7.3 Threshold Optimisation

The production threshold was shifted from the naive 0.5 boundary to **0.3660** using the Youden-J index maximisation on the validation curve. This tighter threshold increases sensitivity toward evasive phishing signatures — particularly the high-reputation-domain attacks identified in the FN analysis — while the precision floor (98.68%) remains above operational acceptability for an institutional mail environment.

8. Discussion

8.1 Security Posture and Fail-Closed Design

Shield Gates is architected to fail closed: any exception in `mail_filter.py`, any unreachable CAPEv2 endpoint, or any model loading failure triggers a DROP verdict rather than a default DELIVER. This is a deliberate security-over-availability trade-off appropriate for high-risk email environments. The `systemd Restart=always` directive ensures rapid service recovery, minimising the duration of any service interruption.

8.2 Computational Overhead

The dominant latency source is CAPEv2 sandbox analysis, which requires 60–120 seconds per attachment. For mail systems with strict delivery SLAs, this necessitates an asynchronous analysis mode where the message is quarantined pending sandbox verdict rather than blocking the SMTP

session. The header and body ML analysis completes in under 300 ms per message on a single-core VM, making the pipeline suitable for moderate-throughput institutional mail environments (hundreds of messages per hour).

8.3 Limitations

- The RoBERTa model requires GPU or significant CPU resources for fine-tuning; the TF-IDF fallback reduces detection quality on zero-day linguistic phishing.
- The current VirtualBox-based sandbox architecture limits throughput to one attachment analysis task at a time per VM.
- High-reputation domain evasion (identified in FN analysis) remains an open challenge without real-time threat intelligence integration.
- The system does not currently process outbound email, missing insider-threat and data-exfiltration scenarios.

9. Future Work

Based on the error analysis and architectural evaluation, the following directions are identified as high-priority for future development:

- **Real-Time URL Intelligence:** Integrate VirusTotal and OpenPhish API lookups within the existing URL analysis pipeline to address high-reputation-domain evasion.
- **Outbound Email Filtering:** Extend `mail_filter.py` to inspect outbound traffic for data leakage and compromised account detection.
- **Containerised Deployment:** Replace the VirtualBox lab environment with Docker Compose and Kubernetes-orchestrated CAPEv2 containers to increase sandbox throughput and simplify deployment across diverse institutional networks.
- **Cross-Platform Extension:** Apply the same header/body/attachment pipeline to messaging platforms (Slack, Teams, WhatsApp Business) via webhook and bot APIs.
- **Adversarial Robustness:** Evaluate the system against adversarially perturbed phishing emails (character substitutions, homograph attacks) and train a dedicated adversarial-example detector.
- **Improved NLP Models:** Fine-tune a multilingual transformer to support Arabic-language phishing campaigns, relevant to the system's intended deployment region.

10. Conclusion

This paper presented **Shield Gates**, a multi-layered secure email gateway that integrates machine learning, natural language processing, and dynamic malware sandboxing into a unified, production-deployable phishing defence system. The gateway is positioned as an inline SMTP proxy anchored by Postfix, with a Python-based filtering layer (`mail_filter.py`) orchestrating three independent detection tracks and a deterministic Decision Engine that aggregates their outputs.

The system was trained and evaluated on a carefully engineered dataset of 14,512 emails, achieving a validation ROC-AUC of 0.9984, a phishing Recall of **96.42%**, and a False Positive Rate of **1.64%** — performance comparable to leading commercial solutions, while remaining fully open-source and reproducible on commodity hardware.

The XGBoost header model identified URL anchor-text mismatch, IP-based URLs, and domain mismatch as the three strongest phishing predictors, underscoring the importance of structural and authentication signals beyond keyword-based filtering. The post-training error analysis surfaced concrete and actionable insights: high-hop mailing-list traffic drives false positives, while high-reputation-domain hosting is the primary false negative vector. These findings directly inform the future work roadmap.

Overall, Shield Gates demonstrates that a comprehensive, academically rigorous email security gateway can be built and deployed on open-source foundations, combining the interpretability of rule-based systems with the power of modern transformer-based NLP and live behavioural sandboxing.

References

- [1] Proofpoint, Inc. (2024). *Proofpoint Email Protection: Classification and Filtering Analysis*. Proofpoint Technical Documentation.
- [2] Abu-Haweleha, R. (2026). *Shield Gates — Secure Email Gateway with NLP Filtering* [Source code]. GitHub. <https://github.com/RamaAbuHaweleha01/MGW>
- [3] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD*, 785–794.
- [4] Liu, Y., et al. (2019). RoBERTa: A Robustly Optimised BERT Pretraining Approach. *arXiv preprint arXiv:1907.11692*.
- [5] Fette, I., Sadeh, N., & Tomasic, A. (2007). Learning to detect phishing emails. *Proceedings of the 16th WWW Conference*, 649–656.
- [6] Cuckoo Foundation. (2022). *CAPEv2: Configurable Analysis and Protection Engine*. <https://github.com/kevoreilly/CAPEv2>
- [7] Kitterman, S. (2014). *RFC 7208: Sender Policy Framework (SPF)*. IETF.
- [8] Crocker, D., Hansen, T., & Kucherawy, M. (2011). *RFC 6376: DomainKeys Identified Mail (DKIM) Signatures*. IETF.
- [9] Kucherawy, M., & Zwicky, E. (2015). *RFC 7489: Domain-based Message Authentication, Reporting, and Conformance (DMARC)*. IETF.
- [10] Klimt, B., & Yang, Y. (2004). The Enron Corpus: A New Dataset for Email Classification Research. *ECML 2004*, 217–226.